

Topic 1.2

Topic 1.2: Objected Oriented Programming

An Object Oriented Programming Language (OOPL) has the properties of *Data encapsulation*, *Polymorphism*, and *Inheritance*:

Data encapsulation hides data from unprivileged agents.

Polymorphism allows the same operators to have multiple meanings. The following example illustrates the use of 4 different addition operators:

```
int    i,j,k;
float  x,y,z;

i = j + k;
z = i + x;
z = x + i;
z = x + y;
```

Inheritance allows us to derive new types from base types, and all the properties of the base type still work.

What are the advantages of using an OOP? Basically, they facilitate team programming. Code is more portable, making it easier to isolate system dependencies in as few places as possible. Software can also be reused more easily, although reusing software with extensions is more difficult, and reusing it with modifications is more difficult still.

There are "religious" arguments as to whether C++ is actually an OOPL. These arguments are very subtle and are beyond the scope of this course. Pragmatically, C++ is generally considered an OOPL by industrial programmers.

Software design methodologies

Numerous software design methodologies have been developed and used down through the years:

Procedural Analysis, as in top-down design, instructs you to focus on the procedures that will manipulate data.

Data flow techniques, such as Yourdan Analysis, which was popular in the late 1970s into the early 1980s, instruct you to identify how the data flows through different parts of a program and between programs in a code system.

A flaw with these and most other design methodologies is that the data is decoupled from the procedures that manipulate the data. In OOD/OOP, we identify the objects which will be used throughout a program and couple the data needed to represent those objects with the procedures that actually perform the manipulation.

to topic 1.3 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-1-dot-3>)